CS-3180: Comparative Languages Lab Problem 5: Data Processing

PURPOSE: The goal of this assignment is to integrate multi-file projects, use higher-order functions, and get a feel for Lisp's **homoiconicity** (where code and data are both represented with S-expressions).

In this lab, you will program an airport passenger screening workflow by ingesting a pre-configured dataset **`Airport.lisp`**, importing your functions from the previous lab **`Contraband.lisp`**, and building a screening and filtering pipeline.

DIRECTIONS: You will write a program called **`Screening.lisp`**. For this lab, you are **not** restricted from defining arbitrary helper functions in the global namespace with **`defun`**. Feel free to break your code into as many logical units as is necessary. However, iterative-style keywords are still prohibited.

File Importing: Your program must use three distinct files in the same working directory. You must include the following lines calls at the very top of your **`Screening.lisp`** source file to access code in the other files:

```
(load "Contraband.lisp")
(load "Airport.lisp")
```

Iterative Keyword Ban:

- You are still **prohibited** from using iterative keywords (**`loop`**, **`dolist`**, **`dotimes`**, **`do`**, etc.). All data structure processing, list iterations, and manifest or luggage traversal must be accomplished using recursion or higher-order functional mapping (this will be easier if you are comfortable with it).

Accessing Provided S-expressions

The data set provided in **`Airport.lisp`** is not a separate string or data-interchange format JSON. In this case, it is instead a collection of native Lisp S-expressions. If you wanted to, you could execute this file directly, although it would not provide any output.

To process individual records within the global data variable **`*passenger-manifest*`**, use association lookups. Every passenger entity follows the shape:

```
(PASSENGER-NAME (DECLARED-ITEMS) (ACTUAL-ITEMS))
```

- To isolate a single passenger sublist, use

```
(assoc 'NAME *passenger-manifest*)
```

- Combined with positional accessors like **`second`** (for declared items) and **`third`** (for actual items), you can extract lists for easy processing. For instance, grabbing an individual's actual luggage items evaluates to:

```
(third (assoc 'SAM *passenger-manifest*)) ; => (JACKET PHONE KNIFE)
```

This is worth practicing a bit, before you dive into programming the filtering logic. Having a solid grasp of processing this data will make writing the main code alot easier.

Screening

Your submission must implement the checks necessary to process a passenger manifest. You must implement functional operations that handle the following criteria:

1. **No-Fly List**: Determine if a passengers name exists in the global `*no-fly-list*`.
2. **X-Ray**: Pull out a passenger's declared items and actual items, pass them to your previously implemented `xray` function checking whether **every** position returned a matching `T` value.
3. **Item Limits**: Cross-reference a passenger's actual belongings against the rules set in `*item-limits*`. If a passenger's items contains an amount of an item **greater than** the allowed limit, they fail validation. You may want to use your `count-items` function from the previous lab.
4. **Contraband**: Recursively scan actual items against `*contraband-list*`. If any illegal item is found, the passenger's bag must be passed through `strip-suitcase` to remove all instances of contraband before boarding.

Desired Output

You must implement a driver program that processes names sequentially through the manifest. Because screening actions require side-effects (printing to standard out), use sequencing keywords like `progn` and `cond` (implicit `progn`) to cleanly map the data.

For each passenger processed, your program must output text according to the table below:

Trigger	Program Output Pattern
No-Fly Trigger	`Reject [NAME] for being on no-fly-list`
X-Ray Mismatch Trigger	`Reject [NAME] for having a bad xray`
Over-Limit Trigger	`Reject [NAME] for exceeding item limit`
Contraband Violation Trigger	`[NAME] was carrying contraband, and has been stripped` `[NAME] boarded the plane with ([CLEANED-ITEMS])`
Safe Clearance / None	`[NAME] boarded the plane with ([ACTUAL-ITEMS])`

Print a newline (`~%`) between passengers to maintain spacing. Your file should execute automatically throught a top level function call that fires when running:

EXAMPLE: Output of your code for the sample data

```
> ./lisp.sh Screening.lisp
ALICE was carrying contraband, and has been striped
ALICE boarded the plane with (JACKET SOCKS LAPTOP)
BOB boarded the plane with (BOOKS PHONE KEYS)
CHARLIE boarded the plane with (SHIRT SOCKS SHOES)
Reject DEREK for being on no-fly-list
Reject GARY for being on no-fly-list
Reject KARLY for being on no-fly-list
ETHAN was carrying contraband, and has been striped
ETHAN boarded the plane with (SHIRT SOCKS)
FIONA was carrying contraband, and has been striped
FIONA boarded the plane with (PHONE LAPTOP)
HANNAH was carrying contraband, and has been striped
HANNAH boarded the plane with (JACKET)
```

```
Reject IAN for exceeding item limit
Reject JULIA for exceeding item limit
Reject KEVIN for exceeding item limit
MIA was carrying contraband, and has been striped
MIA boarded the plane with (PHONE SOCKS)
Reject OLIVER for exceeding item limit
Reject SAM for having a bad xray
TOM was carrying contraband, and has been striped
TOM boarded the plane with (SOCKS LIGHTER)
All passengers processed
```

RUBRIC:

(80 pts) **Base Functionality**

- [5] Each file loads and imports properly
- [15] No-Fly checks and X-Ray identify correct passengers
- [20] Item limits are verified correctly
- [20] Contraband items are identified and removed correctly
- [20] Driver executes and formats passenger processing properly (alliteration :))

(20 pts) **Style**

- [5] Program adheres to looping keyword bans and global function restrictions
- [15] Completed lab report in Markdown format

IMPORTANT NOTES:

- **Any submission containing iterative looping keywords (loop, dolist, etc.) will receive an automatic 50% penalty.**
- **Submissions that do not compile will receive a zero**
- **Submissions with improperly cited AI will receive a zero and an academic integrity violation**
- **Submissions that are partially or fully copied from another submission will receive a zero and an academic integrity violation**